

DODO PAYMENTS

Security & DevOps Engineer

Technical Assessment

Confidential • For Candidate Use Only • 2–3 Days

Overview

You've just joined Dodo Payments — a global Merchant of Record processing card payments across 190+ countries. A fast-moving team shipped `ledger-api`, a microservice handling cardholder-adjacent data, onto a shared cluster with plaintext secrets, a root container, and no network policy. It's in **PCI DSS** scope, and an audit is coming.

Your mission: **harden it end to end, prove your controls work, and then put on the attacker hat.** This assessment spans workload hardening, secure delivery, zero-trust networking, and hands-on reconnaissance & penetration testing. There's no single right answer — we care how you reason about risk and defend your choices.

Role	Security & DevOps Engineer
Company	Dodo Payments
Duration	2–3 days (recommended)
Tasks	4 hands-on tasks
Environment	Runs fully local — a local Kubernetes cluster + GitHub. No cloud account required. Free-tier cloud is optional if you prefer it.
Submission	Public GitHub repository + short write-up

General Guidelines

- Use a **public GitHub repository** for all code, manifests, and documentation.
- Everything runs free and local: a cluster on **kind / k3d / Minikube** + **GitHub Actions**. No AWS/GCP/Azure account is needed to complete any task.
- Provide a clear **README per task** explaining your approach and design decisions.
- Include an **architecture diagram** where it helps (draw.io, Excalidraw, or similar).

- **Show it working** — screenshots, terminal recordings, or pipeline run links are how we verify.
 - **Bonus tasks** are optional but strongly encouraged — they're where you show depth.
-

Task 1 — Deploy & Harden the Workload

Deploy ledger-api (<https://github.com/bhabani-dodo/ledger-api-assignment>) and turn it from insecure to production-grade.

- Deploy ledger-api plus at least one **neighbour service** with Deployments, Services, ConfigMaps, and Ingress.
- Lock down the securityContext: **non-root, read-only root filesystem, all capabilities dropped**, seccomp RuntimeDefault.
- Set **resource requests/limits** and **liveness/readiness probes** on every container.
- Give the service a **dedicated least-privilege ServiceAccount** (drop the default SA) and RBAC scoped to what it actually needs.
- Move secrets **out of git** into a real store — **Sealed Secrets, SOPS+age, or External Secrets** (the plaintext key must be gone).
- Enforce guardrails with **Kyverno or OPA/Gatekeeper**: reject containers that run as root, use :latest, or are unsigned.

Bonus - RBAC roles for developer / operator / admin personas with least privilege. - Pod Security Standards (restricted) enforced at the namespace. - Demonstrate the admission policy **rejecting** the original insecure Deployment. Just show off your security skills.

Task 2 — Secure CI/CD Pipeline & Supply Chain

Rebuild the delivery path so security is enforced by the pipeline, not by good intentions.

- Build a **GitHub Actions** pipeline that builds, scans, signs, and deploys the image (free GitHub runners + GHCR — no cloud account).
- Wire in security gates: **SAST** (Semgrep), **dependency/CVE scan** (Trivy/Grype), **image scan**, and **secrets scan** (gitleaks).
- **Sign the image and generate provenance** with Cosign (keyless) + an SLSA-style attestation.
- State each gate's **fail policy** — what hard-blocks, what warns, and how you handle a CVE with no fix yet.
- Adopt **GitOps with ArgoCD (or Flux)** as the source of truth; show **drift detection + self-heal** after a manual `kubectl edit`.

Bonus - Upload scanner results as **SARIF** so they surface in the repo's Security tab. - cosign verify output proving your image was signed by your workflow. - Canary or blue-green rollout strategy.

Task 3 — Service Mesh & Zero-Trust (Istio)

Build your own service mesh and enforce identity-based zero-trust between services.

- Install **Istio** (`istioctl install`) and bring `ledger-api` + its neighbour into the mesh.
- Enforce **mTLS STRICT** with `PeerAuthentication` — prove a plaintext (non-mTLS) request is **refused** (`istioctl authn tls-check`).
- Write a **default-deny** `AuthorizationPolicy` plus explicit allows keyed on **workload identity (SPIFFE / service account)**, not IP — show an unauthorised service **blocked** and an authorised one **allowed**.
- Explain how workload **certificates are issued and rotated**, and what the trust root is.
- Layer a Kubernetes **NetworkPolicy** (default-deny + explicit allows) underneath for defence-in-depth, and explain what each layer catches that the other doesn't.

Bonus - Expose the service through an **Istio Ingress Gateway** with TLS termination. - A canary release via `VirtualService` + `DestinationRule`. - Tie the mesh boundary back to the **PCI cardholder-data-environment (CDE)** scope.

Task 4 — Reconnaissance & Penetration Testing

Switch sides. First map an attack surface like an outside attacker would, then prove your exploitation skills on an authorized target — all black-box, **no credentials required**.

Part A — Reconnaissance (OSINT, passive)

Using only **public information**, enumerate and map the external attack surface of `dodopayments.tech`.

- Discover subdomains via certificate-transparency logs (**crt.sh**), DNS, and tooling (**subfinder**, **amass**, **assetfinder**).
- Fingerprint live hosts and technologies (**httpx**, **whatweb**) and review public TLS posture (**testssl.sh**).
- Produce an **attack-surface report**: an inventory of what's publicly exposed, the tech behind it, and your **risk observations** — what a real attacker would find interesting and why.

Passive recon only. Part A is limited to public data (DNS, CT logs, HTTP banners, TLS config). **Do not launch active attacks, exploitation, fuzzing, or**

automated scanners against any dodopayments.tech host. Active testing happens only in Part B, against the designated target.

Part B — Penetration Test (authorized target)

Perform a focused web-application penetration test against the **designated vulnerable target** — either the lab host named in your invite (e.g. `example.dodopayments.tech`) or the vulnerable app bundled in the starter repo, run locally. This target is **explicitly authorized** for active testing.

- **Test** for **OWASP Top 10** class issues: broken access control / IDOR, injection (SQLi/XSS), SSRF, security misconfiguration, secrets exposure, auth/session flaws. Tools: Burp Suite Community / OWASP ZAP, nuclei, ffuf, sqlmap.
- **Report** like a professional: executive summary, methodology, and **per finding** — **severity (CVSS v3.1 vector + score)**, affected endpoint, **reproduction steps / PoC** (request-response or screenshot), impact, and **remediation**. Rank by risk.

Rules of engagement. Active testing is permitted **only** against the designated target above. Every production or other `dodopayments.tech` / `dodopayments.com` host and any third-party service is **strictly out of scope**. No DoS, stress, or social engineering. Rate-limit your tools. Testing outside this scope is unauthorised and disqualifying.

Bonus - Chain two findings into a higher-impact attack path. - A retest section showing a finding is closed after remediation. - Map each finding back to the defensive controls from Tasks 1–3 that would have stopped it.

Depth, accuracy, and quality of remediation beat raw finding count — false positives cost you.

Evaluation Criteria

Criteria	What we look for
Security Hardening	Least-privilege, secrets hygiene, admission guardrails, defense-in-depth
Zero-Trust & Networking	Working mTLS, identity-based authz, segmentation that actually enforces
Secure Delivery	Clean CI/CD + GitOps with real, gating security scans and image signing
Offensive Skill	Thorough recon, reproducible findings, accurate CVSS, actionable fixes, disciplined scope

Criteria	What we look for
Automation & Quality	Reproducible, well-documented, production-grade code; minimal manual steps
Judgement	Threat-driven prioritisation, clear trade-offs, defensible decisions

Submission Instructions

- Push all code, manifests, and docs to a **public GitHub repository**, one folder per task.
- Include a **top-level README** summarising your approach and linking each task.
- Provide **screenshots or recordings** of working deployments, pipeline runs, mesh policies enforcing, recon output, and pen-test PoCs.
- Deliver the **recon + pen-test report** (Task 4) as a standalone PDF/Markdown.
- Share the repository link with the hiring team at Dodo Payments.

Important Notes

- The entire assessment **runs locally with free tooling** — no cloud account required. Use free-tier cloud only if you prefer to.
- If you can't finish everything, **prioritise quality over quantity** and document what you'd do with more time.
- Use any tools you'd normally use, including AI assistants — be ready to **defend every artifact and finding** in the follow-up conversation.
- Questions? Reach out to the hiring team at Dodo Payments.

Good luck — we're looking forward to seeing how you think.